

3.1.1.

```
#include <stdio.h>
```

```
// Global declarations
float distance = 120.5;    // in kilometers
float costPerKm = 15.0;    // cost per kilometer
int numTravellers = 4;    // number of people

// Function with no arguments and no return
void calculateTripCost() {
    float totalCost = distance * costPerKm * numTravellers;
    printf("Total cost of the trip: ₹%.2f\n", totalCost);
}

int main() {
    calculateTripCost(); // Function call
    return 0;
}
```

3.1.2

```
#include <stdio.h>
```

```
// Global declarations
int unitsConsumed = 350;    // Units consumed in a month
float ratePerUnit = 6.75;    // Rate per unit in ₹

// Function with no arguments, returns total bill
float calculateBill() {
    float total = unitsConsumed * ratePerUnit;
    return total;
}

int main() {
    float billAmount = calculateBill(); // Function call
    printf("Estimated Monthly Electricity Bill: ₹%.2f\n", billAmount);
    return 0;
}
```

3.1.3

```
#include <stdio.h>
```

```
// Function that accepts score and prints grade
void printGrade(int score) {
    if (score >= 90) {
        printf("Grade: A\n");
    } else if (score >= 80) {
        printf("Grade: B\n");
    } else if (score >= 70) {
        printf("Grade: C\n");
    } else {
        printf("Grade: F\n");
    }
}
```

```

int main() {
    int studentScore;

    // Input from user
    printf("Enter student's score: ");
    scanf("%d", &studentScore);

    // Function call
    printGrade(studentScore);

    return 0;
}

```

3.1.4

```
#include <stdio.h>
```

```

// Function to calculate interest
float calculateInterest(float principal, float rate, int time) {
    float interest = (principal * rate * time) / 100;
    return interest;
}

int main() {
    float principal, rate, earnedInterest;
    int time;

    // Input from user
    printf("Enter principal amount: ₹");
    scanf("%f", &principal);

    printf("Enter rate of interest (%% per annum): ");
    scanf("%f", &rate);

    printf("Enter time (in years): ");
    scanf("%d", &time);

    // Function call
    earnedInterest = calculateInterest(principal, rate, time);

    // Display result
    printf("Interest earned on fixed deposit: ₹%.2f\n", earnedInterest);

    return 0;
}

```

3.2.1.

```
#include <stdio.h>

// Function to print student names and marks
void printMarks(char names[][50], int marks[], int size) {
    for (int i = 0; i < size; i++) {
        printf("Student %d: %s - Marks: %d\n", i + 1, names[i], marks[i]);
    }
}

int main() {
    // Array of student names
    char names[5][50] = {
        "Aarav",
        "Diya",
        "Karthik",
        "Meera",
        "Rohan"
    };

    // Corresponding marks
    int marks[5] = {78, 85, 92, 67, 88};

    // Print names and marks
    printMarks(names, marks, 5);

    return 0;
}
```

3.2.2.

```
#include <stdio.h>

float calculateTotal(float prices[], int count) {
    float total = 0;
    for (int i = 0; i < count; i++) {
        total += prices[i];
    }
    return total;
}

int main() {
    float prices[5];
    for (int i = 1; i < 5; i++)
    {
        printf("%d item price", i);
        scanf("%f", &prices[i]);
    }
}
```

```

float bill = calculateTotal(prices, 5);
printf("Total Bill: ₹%.2f\n", bill);
return 0;
}

```

3.2.3

```

#include <stdio.h>

// Define the Student structure
struct Student {
    char name[50];
    int roll;
    char branch[30];
    float cgpa;
};

// Function to print details of multiple students
void printStudentDetails(struct Student s[], int count) {
    for (int i = 0; i < count; i++) {
        printf("\nStudent %d:\n", i + 1);
        printf("Name   : %s\n", s[i].name);
        printf("Roll No: %d\n", s[i].roll);
        printf("Branch : %s\n", s[i].branch);
        printf("CGPA  : %.2f\n", s[i].cgpa);
    }
}

int main() {
    // Declare and initialize array of students
    struct Student students[3] = {
        {"Anjali", 101, "CSE", 8.75},
        {"Ravi",   102, "ECE", 8.40},
        {"Meera",  103, "ME",  9.10}
    };

    // Call the function with array and count
    printStudentDetails(students, 3);

    return 0;
}

```

3.2.4.

```

#include <stdio.h>

// Define the Employee structure
struct Employee {
    char name[50];
    int id;
    float salary;
    int rating;
};

// Function to update salary based on rating
struct Employee updateSalary(struct Employee e) {
    if (e.rating == 5)
        e.salary *= 1.20;
}

```

```

    else if (e.rating == 4)
        e.salary *= 1.10;
    // No change for rating ≤ 3
    return e;
}

int main() {
    // Declare and initialize array of employees
    struct Employee empList[3] = {
        {"Rahul", 1001, 50000.0, 5},
        {"Sneha", 1002, 45000.0, 4},
        {"Arjun", 1003, 40000.0, 3}
    };

    // Update and display salaries
    for (int i = 0; i < 3; i++) {
        struct Employee original = empList[i];    // Preserve original for comparison
        empList[i] = updateSalary(empList[i]);    // Update salary

        printf("\nEmployee %d:\n", i + 1);
        printf("Name      : %s\n", empList[i].name);
        printf("ID       : %d\n", empList[i].id);
        printf("Initial Salary : ₹%.2f\n", original.salary); // Show original salary
        printf("Updated Salary : ₹%.2f\n", empList[i].salary); // Show updated salary
        printf("Rating      : %d\n", empList[i].rating);
    }

    return 0;
}

```

Additional:

```
#include <stdio.h>
```

```
struct Student
```

```
{  
    int roll;  
    char name[30];  
    float marks1;  
    float marks2;  
    float marks3;  
    float average;  
};
```

```
/* No argument and no return value */
```

```
void displayTitle()
```

```
{  
    printf("\n=====\\n");  
    printf("    STUDENT MANAGEMENT SYSTEM\\n");  
    printf("=====\\n");  
}
```

```
/* No argument and return value */
```

```
struct Student inputStudent()
```

```
{  
    struct Student s;  
  
    printf("\nEnter Roll Number : ");  
    scanf("%d", &s.roll);  
  
    printf("Enter Name      : ");  
    scanf(" %[^\\n]", s.name);  
  
    printf("Enter Marks 1   : ");  
    scanf("%f", &s.marks1);  
  
    printf("Enter Marks 2   : ");  
    scanf("%f", &s.marks2);  
  
    printf("Enter Marks 3   : ");  
    scanf("%f", &s.marks3);  
  
    s.average = 0;  
  
    return s;  
}
```

```
/* Argument and return value */
```

```
float calculateAverage(struct Student s)
```

```
{  
    return (s.marks1 + s.marks2 + s.marks3) / 3;  
}
```

```
/* Argument and no return value */
```

```
void displayStudent(struct Student s)
```

```
{  
    printf("\nRoll Number : %d", s.roll);
```

```

    printf("\nName      : %s", s.name);
    printf("\nMarks 1   : %.2f", s.marks1);
    printf("\nMarks 2   : %.2f", s.marks2);
    printf("\nMarks 3   : %.2f", s.marks3);
    printf("\nAverage   : %.2f\n", s.average);
}

/* Passing array of structures */
void displayAll(struct Student s[], int n)
{
    int i;

    printf("\n===== \n");
    printf("Roll\tName\t\tM1\tM2\tM3\tAverage\n");
    printf("===== \n");

    for(i = 0; i < n; i++)
    {
        printf("%d\t%-15s %.2f\t%.2f\t%.2f\t%.2f\n",
            s[i].roll,
            s[i].name,
            s[i].marks1,
            s[i].marks2,
            s[i].marks3,
            s[i].average);
    }
}

/* Passing structure using pointer */
void updateMarks(struct Student *s)
{
    s->marks1 = s->marks1 + 5;
    s->average = calculateAverage(*s);
}

/* Returning pointer to structure */
struct Student* getStudent(struct Student s[])
{
    return s;
}

int main()
{
    struct Student s[3];
    struct Student *p;
    int i;

    displayTitle();

    printf("\nEnter details of 3 students:\n");

    for(i = 0; i < 3; i++)
    {
        printf("\n--- Student %d ---\n", i + 1);

        s[i] = inputStudent();

        s[i].average = calculateAverage(s[i]);
    }
}

```

```
}

printf("\n\nBEFORE UPDATING MARKS");
displayAll(s, 3);

printf("\n\nAdding 5 bonus marks to Student 1...\n");

updateMarks(&s[0]);

printf("\n\nAFTER UPDATING MARKS");
displayAll(s, 3);

p = getStudent(s);

printf("\n\nSTUDENT RETURNED USING POINTER");
printf("\nRoll Number : %d", p->roll);
printf("\nName      : %s", p->name);
printf("\nMarks 1    : %.2f", p->marks1);
printf("\nAverage   : %.2f\n", p->average);

printf("\n\nPASSING COMPLETE STRUCTURE");
displayStudent(s[1]);

return 0;
}
```